UNIVERSIDAD RAFAEL LANDIVAR

FACULTAD DE INGENIERIA

LENGUAJES FORMALES Y AUTOMATAS

ING. ROJAS ARGUETA CESAR SALVADOR

Documento de análisis y diseño

Proyecto 2

INTEGRANTES:

RODRIGO RUIZ 1037623

JAROD BOLAÑOS 1251621

CIUDAD DE GUATEMALA

Indice

Propósito 3
Descripción general de la solución 3
Glosario de términos y clases y estructura de los datos 4
Mapa por opción del menú 6
Expresiones regulares 8
Algoritmo de construcción de subconjuntos 9
Aclaraciones de variables y funcionamiento 10
automata.py 10
cargador.py 11
cargador_afnd.py 11
validador.py 12
validador_afnd.py 13
conversor.py 13
simulador.py 14
colores.py 14
main.py 14
Restricciones 16
Restricciones generales del proyecto 16
Reglas de acceso entre opciones del menú 16
Diagramas de flujo del proyecto 18
Diagrama principal 18
Diagrama Crear un AFD manualmente 19
Diagrama Cargar un AFD desde archivo .txt 20
Diagrama Crear un AFND manualmente 21
Diagrama Cargar un AFND desde archivo .txt 22
Diagrama Mostrar la definición formal y la tabla del autómata cargado 23
Diagrama Validar la estructura del autómata 24
Diagrama Convertir el AFND cargado en un AFD equivalente 26
Diagrama Mostrar la tabla de equivalencias de macroestados 27

Diagrama Mostrar la tabla de transición del AFD generado 28
Diagrama Evaluar una cadena 29
Diagrama Evaluar un archivo de cadenas 30
Diagrama Consultar el historial de evaluaciones 31
Diagrama Realizar el análisis estructural 32
Diagrama Cargar o crear otro autómata 33
Diagrama Salir 34
Casos de prueba 34

Propósito

El propósito de este proyecto es ampliar el motor de autómatas desarrollado en la Fase 1 para que, además de crear, cargar, validar y simular Autómatas Finitos Deterministas (AFD), permita crear y cargar Autómatas Finitos No Deterministas (AFND), validarlos, convertirlos en un AFD equivalente mediante el algoritmo de construcción de subconjuntos y aplicar sobre el resultado todas las herramientas de la Fase 1: definición formal, tabla de transición, evaluación de cadenas con traza, evaluación por lotes, historial y análisis estructural. Además, cada cadena evaluada sobre el AFD generado se simula también directamente sobre el AFND original, para comprobar que ambos reconocen el mismo lenguaje.

Descripción general de la solución

La solución se organiza en un menú de 15 opciones que evoluciona las 10 opciones de la Fase 1 sin eliminar ninguna. Las opciones son las siguientes:

1. Crear un AFD manualmente.
2. Cargar un AFD desde un archivo .txt.
3. Crear un AFND manualmente.
4. Cargar un AFND desde un archivo .txt.
5. Mostrar la definición formal y la tabla del autómata cargado.
6. Validar la estructura del autómata.
7. Convertir el AFND cargado en un AFD equivalente.
8. Mostrar la tabla de equivalencias de macroestados.
9. Mostrar la tabla de transición del AFD generado.
10. Evaluar una cadena.

11. Evaluar un archivo de cadenas.
12. Consultar el historial de evaluaciones.
13. Realizar el análisis estructural.
14. Cargar o crear otro autómata.
15. Salir.

A continuación se explica qué partes conforman cada opción y cómo funcionan. Primero se describen las clases y estructuras de datos; luego, cada opción del menú, las expresiones regulares, el algoritmo de conversión y, por último, el detalle de cada módulo del código.

Glosario de términos y clases y estructura de los datos

Término	Dónde vive	Qué es
AFD	automata.py	Quíntupla M=(Q,Σ,δ,q0,F). delta: dict[(estado,simbolo)] -> str (un solo destino, o ausente si está incompleto).
AFND	automata.py	Misma quíntupla, pero delta: dict[(estado,simbolo)] -> frozenset[str] (0, 1 o varios destinos).
Macroestado	Concepto, no una clase	Un subconjunto (frozenset) de estados del AFND. Cada macroestado alcanzado durante la conversión se convierte en UN estado del AFD generado.
Etiqueta	conversor._siguiente_etiqueta	Nombre legible que se le asigna a un macroestado, en el orden en que se descubre: A, B, C, ..., Z, AA, AB, ...
tabla_equivalencias	Variable de sesión en main.py; la arma conversor.construir_afd_desde_afnd	dict[etiqueta] -> frozenset (el macroestado que esa etiqueta representa). Es lo que muestra la opción 8.
automata_cargado	Variable de sesión en main.main()	El AFD o AFND recién creado/cargado con las opciones 1-4. Se distingue con isinstance.

afd_generado	Variable de sesión en main.main()	El AFD que resulta de convertir automata_cargado (opción 7). None mientras no se convierta.
historial	Instancia de Historial (clase en main.py)	Registro de cadenas evaluadas contra el autómata activo (cadena, veredicto, traza, motivo y, si aplica, veredicto del AFND). Se reinicia en las opciones 1-4 y 14.
ResultadoEvaluacion / Paso	Dataclasses en simulador.py	Resultado de evaluar una cadena (cadena, traza, veredicto, motivo) y cada paso de la traza (estado_origen, simbolo, estado_destino).
AnalisisEstructural	Dataclass en validador.py	alcanzables, inaccesibles, finales_alcanzables, lenguaje_vacio: resultado del análisis estructural (opción 13, y automático tras validar un AFD).
obtener_afd_operable	Función en main.py	Devuelve el AFD para las opciones 10, 11 y 13: automata_cargado si ya es un AFD validado, o afd_generado si automata_cargado es un AFND ya convertido.
errores_carga	Atributo de AFD y AFND (automata.py)	Lista de errores de lectura del archivo de origen. Si no está vacía, el autómata se construyó de forma parcial y la opción 6 no lo marca como válido.
PasoAFND / ResultadoEvaluacionAFND	Dataclasses en simulador.py	Paso de la simulación directa del AFND (conjunto_origen, simbolo, conjunto_destino) y su resultado (cadena, traza, veredicto, motivo). Se usan para comparar el veredicto del AFND con el del AFD generado.
afnd_original	Función en main.py	Devuelve el AFND cargado (o None si es un AFD) para que las opciones 10 y 11 comparen su veredicto con el del AFD generado.

Mapa por opción del menú

Opción	Módulo / función	Clases	Datos de entrada	Restricciones clave
1. Crear un AFD manualmente	cargador.crear_afd_manual() (usa _pedir_conjunto, _pedir_conjunto_final, _pedir_transiciones)	AFD	Nombre; Q; Σ; q0; F (puede ir vacía); transiciones origen,simbolo,destino hasta FIN	Nombre no vacío; Q/Σ sin vacíos ni duplicados; Σ sin épsilon; q0∈Q; F⊆Q; origen/destino∈Q, símbolo∈Σ; (origen,símbolo) sin repetir ("sería un AFND")
2. Cargar un AFD desde .txt	cargador.cargar_afd_desde_archivo(ruta) (+ normalizar_ruta_archivo)	AFD	Ruta del archivo	Cada línea debe calzar con un PATRON_*; épsilon prohibido; (origen,símbolo) repetido = error; TIPO= distinto de AFD o destinos con | indican que es un AFND; deben existir NOMBRE, ESTADOS, ALFABETO, INICIAL; los errores se guardan en errores_carga
3. Crear un AFND manualmente	cargador_afnd.crear_afnd_manual() (+ _pedir_transiciones_afnd; reutiliza funciones de cargador.py)	AFND	Igual que la 1, pero transición origen,simbolo,destino1|destino2|... (∅ o vacío = sin destino)	Igual que la 1, salvo que un (origen,símbolo) admite varios destinos si se declaran juntos con |; declararlo dos veces por separado es error
4. Cargar un AFND desde .txt	cargador_afnd.cargar_afnd_desde_archivo(ruta)	AFND	Ruta del archivo	PATRON_TRANSICION_AFND y PATRON_DESTINOS_AFND (rechaza q0|, q0||q1, q0,q1 y destinos repetidos); TIPO= debe decir AFND; ∅ o vacío = sin destino; épsilon prohibido ("no forma parte del alcance de esta fase"); los errores se guardan en errores_carga

5. Mostrar definición formal y tabla	main.accion_mostrar_automata_cargado despacha por isinstance a funciones de AFD o AFND	AFD o AFND	Ninguno (usa automata_cargado)	Requiere automata_cargado is not None
6. Validar la estructura	main.accion_validar_automata_cargado → accion_validar_afd o accion_validar_afnd (ambas llaman antes a reportar_errores_de_carga); si ya hubo conversión, también verificar_afd_generado	AFD/AFND, AnalisisEstructural	Solo AFD incompleto: S/N para agregar estado de trampa	Si errores_carga no está vacía, el autómata NO es válido. AFD: q0∈Q, F⊆Q, transiciones consistentes, completitud (o trampa). AFND: q0∈Q, F⊆Q, destinos⊆Q; no existe "incompleto"
7. Convertir AFND en AFD	main.accion_convertir → conversor.construir_afd_desde_afnd → main.verificar_afd_generado (validador.validar_estructura de la Fase 1)	AFND (entrada), AFD (salida), tabla_equivalencias	Ninguno	automata_cargado debe ser AFND y estar validado; construcción de subconjuntos; el AFD generado debe pasar la validación determinista y se muestra su análisis estructural
8. Tabla de equivalencias	main.accion_mostrar_equivalencias(tabla_equivalencias, afd_generado)	—	Ninguno	Requiere tabla_equivalencias (opción 7 exitosa); indica macroestado inicial, finales y el estado de trampa ∅
9. Tabla del AFD generado	main.accion_mostrar_definicion y main.accion_mostrar_tabla con afd_generado (las mismas de la opción 5)	AFD	Ninguno	Requiere afd_generado is not None
10. Evaluar una cadena	main.accion_evaluar_cadena → evaluar_y_mostrar → simulador.evaluar_cadena e imprimir_traza; si el origen es un AFND, además simulador.evaluar_cadena_afnd, imprimir_traza_afnd e imprimir_comparacion	ResultadoEvaluacion, Paso, ResultadoEvaluacionAFND, PasoAFND, Historial	Cadena (vacía = λ)	Requiere obtener_afd_operable(...) is not None; si el origen es un AFND se comparan los veredictos del AFND y del AFD
11. Evaluar un archivo de	main.accion_evaluar_archivo → evaluar_y_mostrar por cada línea; simulador.evaluar_lote para el resumen	ResultadoEvaluacion, ResultadoEvaluacio	Ruta del archivo (una cadena por línea)	Igual que la 10, aplicado a cada línea; el resumen indica cuántas cadenas coinciden entre AFND y AFD

cadenas		nAFND, Historial		
12. Consultar historial	Historial.mostrar()	Historial, RegistroHistorial	Ninguno	Muestra veredicto, traza, motivo y, si aplica, el veredicto del AFND; si está vacío solo informa; se reinicia con 1-4 y 14
13. Análisis estructural	main.imprimir_analisis_estructural validador.analizar_estructura	AnalisisEstructural	Ninguno	Misma guardia que 10/11 (obtener_afd_operable)
14. Cargar o crear otro autómata	Bucle de main(), rama opcion == 14 → main.accion_cargar_o_crear_otro	—	Opción 0-4 (0 = volver al menú)	Reinicia automata_cargado, afd_generado, tabla_equivalencias e historial y permite crear o cargar el siguiente autómata sin reiniciar el programa
15. Salir	Bucle de main(), break	—		

Expresiones regulares

Patrón	Módulo	Qué reconoce
PATRON_NOMBRE	cargador.py / cargador_afnd.py	Línea NOMBRE=... → ^NOMBRE=(.+)$
PATRON_TIPO	cargador.py / cargador_afnd.py	Línea opcional TIPO=... → ^TIPO=(.+)$ (en la opción 2 debe decir AFD y en la opción 4 debe decir AFND)
PATRON_ESTADOS	Ambos	Línea ESTADOS=... → ^ESTADOS=(.+)$
PATRON_ALFABETO	Ambos	Línea ALFABETO=... → ^ALFABETO=(.+)$
PATRON_INICIAL	Ambos	Línea INICIAL=... → ^INICIAL=(.+)$
PATRON_FINALES	Ambos	Línea FINALES=... (puede ir vacía) → ^FINALES=(.*)$
PATRON_ENCABEZADO_TRANSICIONES	Ambos	La línea literal TRANSICIONES: → ^TRANSICIONES:$
PATRON_TRANSICION	cargador.py	origen,simbolo,destino (un solo destino, AFD) → ^([A-Za-z0-9_]+)\s*,\s*([^\s,]+)\s*,\s*([A-Za-z0-9_]+)$
PATRON_TRANSICION_AFND	cargador_afnd.py	origen,simbolo,destinos (AFND); el tercer grupo es el resto de la línea y luego se valida con PATRON_DESTINOS_AFND →

		^([A-Za-z0-9_]+)\s*,\s*([^\s,]+)\s*,\s*(.*)$
PATRON_DESTINOS_AFND	cargador_afnd.py	Columna de destinos de un AFND: ∅, vacío, un estado o varios separados por | (rechaza q0|, q0||q1 y q0,q1) → ^(?:∅|[A-Za-z0-9_]+(?:\s*|\s*[A-Za-z0-9_]+)*)?$
PATRON_IDENTIFICADOR	cargador.py (importado por cargador_afnd.py)	Nombre de estado válido → ^[A-Za-z0-9_]+$

Algoritmo de construcción de subconjuntos

Paso	Qué hace	Línea / idea clave del código
1	Macroestado inicial	frozenset({afnd.q0}) (conversor.py, línea 68)
2	Cola de pendientes	deque([macroestado_inicial]) (línea 72), procesada con popleft() (línea 76): recorrido en anchura (BFS)
3	Para cada macroestado y cada símbolo	destino |= afnd.delta.get((estado, simbolo), frozenset()) para cada estado del macroestado (líneas 77-80)
4	¿Macroestado nuevo?	if destino not in etiquetas: se le asigna la siguiente etiqueta y se encola (líneas 82-85)
5	Se registra la transición del AFD	delta_afd[(etiquetas[actual], simbolo)] = etiquetas[destino] (línea 87)
6	Termina cuando la cola queda vacía	while pendientes (línea 75): al vaciarse la cola ya se descubrieron todos los macroestados alcanzables
7	Estados finales del AFD	if subconjunto & afnd.F (líneas 90-94): basta con un estado final del AFND
8	Resultado	Un AFD + tabla_equivalencias (líneas 96-107); main.verificar_afd_generado lo pasa por validador.validar_estructura (Fase 1) para confirmar que es determinista y completo

Ejemplo resuelto con ejemplos/afnd_dos_destinos.txt (Σ = {a, b}, q0 = q0, F = {q2}, δ(q0,a) = {q0, q1}, δ(q0,b) = {q0}, δ(q1,a) = ∅, δ(q1,b) = {q2}, δ(q2,a) = δ(q2,b) = {q2}). Partiendo de A = {q0}, cada fila calcula la unión de destinos de los estados del macroestado para cada símbolo; cada subconjunto nuevo recibe la siguiente etiqueta y se procesa después. La cola se vacía con 4 macroestados:

Macroestado	Subconjunto del AFND	Con a	Con b	¿Final?
A (inicial)	{q0}	{q0} ∪ ... = {q0, q1} → B (nuevo)	{q0} → A	No

Macroestado	Subconjunto del AFND	Con a	Con b	¿Final?
B	{q0, q1}	{q0, q1} ∪ ∅ = {q0, q1} → B	{q0} ∪ {q2} = {q0, q2} → C (nuevo)	No
C	{q0, q2}	{q0, q1} ∪ {q2} = {q0, q1, q2} → D (nuevo)	{q0} ∪ {q2} = {q0, q2} → C	Sí (contiene q2)
D	{q0, q1, q2}	{q0, q1} ∪ ∅ ∪ {q2} → D	{q0} ∪ {q2} ∪ {q2} → C	Sí (contiene q2)

Como ningún macroestado alcanzado es vacío, este AFND no genera estado de trampa; en cambio, ejemplos/afnd_vacio.txt sí alcanza ∅ (δ(r0,b) = ∅), que se convierte en el macroestado B y regresa a sí mismo con cada símbolo. El mismo resultado se obtiene en el programa con las opciones 7, 8 y 9.

Aclaraciones de variables y funcionamiento

automata.py

La clase AFD (líneas 23-77) conserva la estructura de la Fase 1. Su constructor (__init__, líneas 55-63) construye la quíntupla del autómata, y la variable central es self.delta = dict(delta) (línea 59), un diccionario que asocia cada par (estado, símbolo) con un único destino. En la línea 62 se establece self.validado = False, de modo que todo autómata nace sin validar, y en la línea 63 se inicializa self.errores_carga = [], la lista donde los cargadores guardan los errores de lectura del archivo. El método transicion (líneas 65-71) implementa la función de transición δ de forma genérica mediante delta.get((estado, simbolo)), lo que evita cualquier cadena de if/elif; si no existe la transición devuelve None, lo que indica que el AFD está incompleto.

La clase AFND (líneas 80-141) es nueva en la Fase 2 y sigue el mismo patrón que AFD, con una diferencia clave. En la línea 121, self.delta = {clave: frozenset(destinos) for clave, destinos in delta.items()} define la estructura de datos más importante de toda la fase: cada valor del diccionario de transiciones es un frozenset y no un str, lo que permite representar cero, uno o varios destinos. El método transicion (líneas 127-135) devuelve self.delta.get((estado, simbolo), frozenset()). El segundo argumento de .get marca la diferencia con el AFD: mientras en este la ausencia de una transición devuelve None y se interpreta como un error de completitud, en el AFND devuelve el conjunto vacío (∅), que es un resultado válido. Esta única línea explica por qué un AFND no tiene transiciones "incompletas". Igual que el AFD, el AFND tiene el atributo errores_carga (línea 125).

cargador.py

Este módulo conserva la lógica de la Fase 1, con mejoras en la lectura de archivos. Entre las líneas 39 y 51 se definen las expresiones regulares (PATRON_NOMBRE hasta PATRON_TRANSICION, además de PATRON_TIPO y PATRON_IDENTIFICADOR), y en la línea 55 el conjunto SIMBOLOS_PROHIBIDOS. La función crear_afd_manual (líneas 71-99) y sus funciones auxiliares _pedir_conjunto (102-117), _pedir_conjunto_final (120-135) y _pedir_transiciones (138-176) siguen un patrón de entrada con ciclo while: cada una solicita un dato, lo valida (sin vacíos, sin duplicados, sin épsilon y, cuando corresponde, como subconjunto de Q) y vuelve a pedirlo si la validación falla.

La función normalizar_ruta_archivo (líneas 183-204) realiza una acción de saneamiento y no de validación: elimina comillas, expande ~ y variables de entorno y unifica los separadores de ruta. Por su parte, cargar_afd_desde_archivo (líneas 207-383) conserva el comportamiento de la Fase 1 y agrega: la detección de campos declarados dos veces (función interna _es_declaracion_nueva, líneas 243-259); un mensaje específico cuando una transición trae varios destinos separados por | (línea 270), porque eso corresponde a un AFND; el reconocimiento de la línea TIPO= (línea 303), que si no dice AFD indica usar la opción 4; y, en la línea 382, la copia de los errores en afd.errores_carga, para que un AFD leído con errores nunca pueda darse por válido.

En la Fase 2, estas funciones son invocadas por main.accion_crear_manual() y main.accion_cargar_archivo(), y su resultado ya no se almacena en una variable afd fija, sino en automata_cargado, que también puede contener un AFND.

cargador_afnd.py

Entre las líneas 20 y 27, este módulo importa desde cargador.py los elementos PATRON_IDENTIFICADOR, SIMBOLOS_PROHIBIDOS, _pedir_conjunto, _pedir_conjunto_final, _separar_lista y normalizar_ruta_archivo, en lugar de duplicarlos, lo que constituye un ejemplo directo de reutilización de código.

Las expresiones regulares se encuentran entre las líneas 36 y 52. PATRON_TIPO (línea 37) reconoce la línea TIPO=; si su valor no es AFND se registra un error. PATRON_TRANSICION_AFND (línea 45) separa la línea en origen, símbolo y columna de destinos; esa columna se valida después con PATRON_DESTINOS_AFND (línea 52), que solo acepta ∅, vacío, un estado o varios estados separados por |, y rechaza separadores sobrantes (q0|, q0||q1), comas (q0,q1) o nombres que no sean identificadores. En la línea 56, TOKENS_SIN_DESTINO = {'∅', ''} establece la convención de qué se considera "sin destino".

La función _separar_destinos (líneas 59-73) transforma el texto leído en el tipo de dato que exige AFND.delta: si el texto no respeta PATRON_DESTINOS_AFND devuelve

None, para que quien la llama reporte el error; si pertenece a TOKENS_SIN_DESTINO devuelve frozenset(); de lo contrario separa por | y construye el frozenset correspondiente. La función _destinos_repetidos (líneas 76-80) detecta estados repetidos en la misma transición (por ejemplo q0|q0), que también se reportan como error.

La función crear_afnd_manual (líneas 87-115) y su auxiliar _pedir_transiciones_afnd (líneas 118-173) reutilizan el patrón de entrada con while. La condición de la línea 149, if (origen, simbolo) in delta, es la que distingue la Fase 2 de la Fase 1: mientras en la primera fase un par repetido convertía al autómata en no determinista, ahora se le indica al usuario que debe declarar juntos todos los destinos separados por |. En la línea 155 se obtienen los destinos con _separar_destinos(...), en la línea 162 se rechazan destinos repetidos y en la línea 166 se verifica que cada destino pertenezca a Q mediante invalidos = [d for d in destinos if d not in Q].

La función cargar_afnd_desde_archivo (líneas 180-351) comparte el esqueleto de cargador.cargar_afd_desde_archivo, con estas diferencias: en la línea 242 se rechaza un par (origen, símbolo) declarado dos veces; en las líneas 249 y 256 se validan los destinos con _separar_destinos y _destinos_repetidos; en la línea 273 se maneja la línea TIPO=; y en la línea 350 los errores se copian en afnd.errores_carga. La pertenencia de cada destino a Q no se verifica al leer el archivo: esa verificación se delega a validador_afnd.py, siguiendo el mismo diseño de cargador.py.

validador.py

Este módulo no tiene cambios de código respecto a la Fase 1. La función validar_estructura (líneas 29-80) verifica que q0 pertenezca a Q, que F sea subconjunto de Q, que las transiciones sean consistentes y detecta las transiciones incompletas. La función analizar_estructura (líneas 83-116) calcula los estados alcanzables mediante un recorrido del grafo, y a partir de ellos obtiene los inaccesibles (Q - alcanzables) y los finales alcanzables (F & alcanzables). La función completar_con_estado_trampa (líneas 119-150) agrega el estado q_trampa junto con sus auto-transiciones.

La novedad en la Fase 2 es que estas funciones se reutilizan en más puntos: main.imprimir_analisis_estructural (líneas 383-396 de main.py) invoca analizar_estructura para la opción 13 y automáticamente después de validar un AFD (accion_validar_afd, líneas 430-466); además, main.verificar_afd_generado (líneas 530-550) usa validar_estructura sobre el AFD generado por la conversión. Es el mismo código de la Fase 1 con varios puntos de entrada, lo que ilustra la reutilización de funciones.

validador_afnd.py

La función validar_estructura_afnd (líneas 19-51) sigue la misma lógica que validador.validar_estructura, pero es más breve. En la línea 32 se verifica que afnd.q0 pertenezca a afnd.Q, y en la línea 35 se calcula faltantes_F = afnd.F - afnd.Q, igual que en el caso del AFD. La diferencia aparece en el ciclo sobre afnd.delta.items() (líneas 39-49): en la línea 44, faltantes_destino = destinos - afnd.Q emplea una resta de conjuntos, ya que destinos es un frozenset que puede contener varios estados, en lugar de comparar una sola cadena contra Q.

Es importante señalar que en este módulo no existe ningún cálculo de transiciones incompletas ni llamada para completar con un estado de trampa, porque un AFND no tiene ese concepto, tal como se desprende de automata.AFND.transicion (línea 135).

conversor.py

Este módulo es completamente nuevo en la Fase 2 y constituye el núcleo de la segunda funcionalidad del proyecto. En la línea 29 se define la constante ALFABETO_ETIQUETAS = "ABCDEFGHIJKLMNOPQRSTUVWXYZ", que sirve de apoyo para nombrar los macroestados. La función _siguiente_etiqueta (líneas 32-43) convierte un índice numérico en una etiqueta al estilo de las columnas de Excel; la línea 41, indice, resto = divmod(indice - 1, 26), es la que explica la secuencia A, B, …, Z, AA, AB. La función formatear_subconjunto (líneas 46-51) decide si un macroestado se imprime como ∅ o como {q0, q1}; la usan main.py (opciones 5 y 8) y simulador.py (traza del AFND).

La función construir_afd_desde_afnd (líneas 54-108) implementa el algoritmo de construcción de subconjuntos. En la línea 68 se define el macroestado inicial como frozenset({afnd.q0}). Entre las líneas 70 y 73 se inicializan las variables del algoritmo: etiquetas (diccionario), orden_descubrimiento (lista), pendientes (una deque que funciona como cola) y delta_afd (diccionario que se va completando). La línea 75 (while pendientes:) junto con la línea 76 (actual = pendientes.popleft()) conforman un recorrido en anchura (BFS). Entre las líneas 77 y 80, dos ciclos anidados, uno por símbolo y otro por cada estado del macroestado actual, calculan la unión de destinos. Entre las líneas 82 y 85 se evalúa si el macroestado resultante es nuevo, en cuyo caso se le asigna una etiqueta y se encola. En la línea 87 se registra la transición correspondiente del AFD generado.

Una vez vaciada la cola, entre las líneas 89 y 94 se calculan Q_afd, formado por todas las etiquetas, y F_afd, formado por los macroestados que intersectan con afnd.F (línea 93, if subconjunto & afnd.F). Entre las líneas 96 y 103 se construye el AFD de salida con la misma clase AFD de la Fase 1. El conversor no marca el resultado como validado: main.verificar_afd_generado lo pasa por validador.validar_estructura para

comprobar explícitamente que tiene exactamente un destino por cada par (macroestado, símbolo). Finalmente, entre las líneas 105 y 107 se arma la tabla_equivalencias.

simulador.py

Las funciones de la Fase 1, evaluar_cadena (líneas 61-101), evaluar_lote (líneas 104-128) e imprimir_traza (líneas 131-153), conservan la misma lógica.

En la Fase 2, estas funciones reciben el AFD que devuelve main.obtener_afd_operable(...) (líneas 110-137 de main.py), el cual puede ser un AFD cargado directamente o el AFD generado a partir de una conversión. El simulador no necesita saber de dónde proviene el autómata, lo que demuestra que reutilizarlo cumple con el principio de compatibilidad establecido en el enunciado.

Lo nuevo de la Fase 2 es la simulación directa del AFND, que se usa para comparar resultados con el AFD generado. Las dataclasses PasoAFND (líneas 161-166) y ResultadoEvaluacionAFND (líneas 170-175) guardan la traza como conjuntos de estados. La función evaluar_cadena_afnd (líneas 178-212) parte del conjunto {q0} (línea 194) y, por cada símbolo, calcula la unión de los destinos de todos los estados activos (líneas 206-207); es decir, sigue todos los caminos posibles a la vez. Al final, la cadena es aceptada si el conjunto activo contiene algún estado final (línea 211). Es la misma unión que realiza el algoritmo de construcción de subconjuntos, pero calculada solo para la cadena evaluada; por eso, si la conversión es correcta, ambos veredictos siempre coinciden. imprimir_traza_afnd (líneas 215-230) muestra la secuencia de conjuntos e imprimir_comparacion (líneas 233-245) indica si los veredictos del AFND y del AFD coinciden.

colores.py

Este módulo no tiene cambios y es un utilitario de presentación basado en códigos ANSI. No aporta acciones, datos ni restricciones propias del dominio de autómatas; su relevancia se limita a las decisiones de diseño, particularmente la elección de implementarlo manualmente en lugar de depender de una biblioteca externa como colorama.

main.py

Este archivo fue reescrito para la Fase 2 y concentra la mayor parte de las decisiones de diseño. Las funciones mostrar_menu (líneas 58-77) y pedir_opcion (líneas 80-94) reciben como dato de entrada la opción del usuario y exigen que sea un entero entre 1 y 15, lo cual se verifica con texto.isdigit() (línea 87) y con la comparación de la línea 91.

La función hay_automata_cargado (líneas 101-107) es una guardia simple que utilizan las opciones 5 y 6.

La función obtener_afd_operable (líneas 110-137) es la pieza central de la compatibilidad entre ambas fases. En la línea 127, la condición isinstance(automata_cargado, AFD) determina si el autómata se devuelve tal cual o si, por tratarse de un AFND, se exige que exista afd_generado (línea 134). Esta función es utilizada por las opciones 10, 11 y 13.

Las estructuras RegistroHistorial (líneas 145-153), una dataclass que guarda la cadena, el veredicto, la traza, el motivo y, si aplica, el veredicto del AFND, e Historial (líneas 156-208), su clase contenedora, gestionan el registro de evaluaciones; el método mostrar (líneas 185-208) implementa la opción 12. Las funciones accion_crear_manual, accion_cargar_archivo, accion_crear_manual_afnd y accion_cargar_archivo_afnd (líneas 215-272) son envoltorios delgados: la lógica real reside en cargador.py y cargador_afnd.py, y aquí solo se añaden los mensajes de éxito, error o carga parcial.

Las funciones accion_mostrar_definicion y accion_mostrar_tabla (líneas 279-318), junto con sus equivalentes para AFND (líneas 321-365), calculan dinámicamente los anchos de columna (líneas 300 y 347) según el contenido más largo, de manera que la tabla se mantenga alineada incluso con estados como q_trampa. La función accion_mostrar_automata_cargado (líneas 368-376) es el primer despachador basado en isinstance (línea 371), patrón que se repite en accion_validar_automata_cargado (líneas 492-502), la cual delega en accion_validar_afd (líneas 430-466) o en accion_validar_afnd (líneas 469-489) y, si ya existe un AFD generado, lo vuelve a validar. Antes de validar, ambas llaman a reportar_errores_de_carga (líneas 399-416): si el autómata se leyó de un archivo con errores, estos se muestran y el autómata queda como no válido; reportar_errores_adicionales (líneas 419-427) lista además los errores estructurales de lo que sí se pudo leer. La función imprimir_analisis_estructural (líneas 383-396) es una acción compartida por las opciones 6, 7 y 13.

La función accion_convertir (líneas 509-527) aplica dos guardias en cascada (líneas 515 y 518) antes de llamar a conversor.construir_afd_desde_afnd (línea 522) y luego invoca verificar_afd_generado (líneas 530-550), que pasa el resultado por validador.validar_estructura (línea 538) y muestra su análisis estructural. La función accion_mostrar_equivalencias (líneas 553-573) presenta la tabla_equivalencias e indica el macroestado inicial, los finales y el estado de trampa ∅. La función evaluar_y_mostrar (líneas 580-594) evalúa la cadena sobre el AFD y, si el autómata cargado es un AFND, también lo simula directamente (línea 590) y compara ambos veredictos (línea 592); la usan accion_evaluar_cadena (líneas 597-601) y accion_evaluar_archivo (líneas 604-635), que al final del lote indica cuántas cadenas coinciden. La función accion_cargar_o_crear_otro (líneas 642-664) implementa la

opción 14 y afnd_original (líneas 667-670) devuelve el AFND cargado para la comparación.

Por último, la función main() (líneas 677-782) declara las variables de sesión (líneas 689-692) y contiene la estructura condicional de 15 ramas (líneas 703-782). En las opciones 1 a 4 (líneas 703-725) y en la 14 (líneas 772-778) se repite un mismo patrón: se reinician afd_generado, tabla_equivalencias e historial, lo que implementa en código la regla de la Fase 2 según la cual, al cargar un nuevo autómata, se limpian los datos anteriores.

Restricciones

Las reglas de validación del AFD son las establecidas en la Fase 1 del proyecto; las del AFND y las restricciones generales son las que pide el enunciado de la Fase 2.

Restricciones generales del proyecto

- Debe funcionar con cualquier AFD/AFND válido, no solo con un caso particular.
- Prohibido usar librerías que implementen autómatas o la construcción de subconjuntos.
- δ debe ser un diccionario genérico, nunca una cadena de if/elif por autómata específico (ver los métodos AFD.transicion y AFND.transicion en automata.py).
- Lectura y validación sintáctica de archivos mediante expresiones regulares.
- Separación de responsabilidades entre carga, validación, conversión y simulación (uno por módulo: cargador*.py, validador*.py, conversor.py, simulador.py).
- Las funciones nuevas se integran al programa de Fase 1, no son una solución aparte.

Reglas de acceso entre opciones del menú

- No se puede convertir (opción 7) sin un AFND válido cargado.
- No se puede evaluar cadenas ni hacer análisis estructural (10, 11, 13) sobre un AFND sin convertir antes.
- Un AFD cargado directamente (opciones 1/2) nunca necesita conversión — puede evaluarse de inmediato tras validarse.
- Un autómata leído de un archivo con errores queda construido de forma parcial: puede consultarse (opción 5), pero la opción 6 no lo marca como válido, por lo que tampoco puede convertirse ni evaluarse.
- Sobre un AFND convertido, las opciones 10 y 11 evalúan la cadena en el AFD generado y además simulan el AFND original para comparar ambos veredictos.

- Al cargar o crear otro autómata (opciones 1-4 y 14) se descartan el AFD generado, la tabla de equivalencias y el historial anteriores.

Diagramas de flujo del proyecto

Diagrama principal

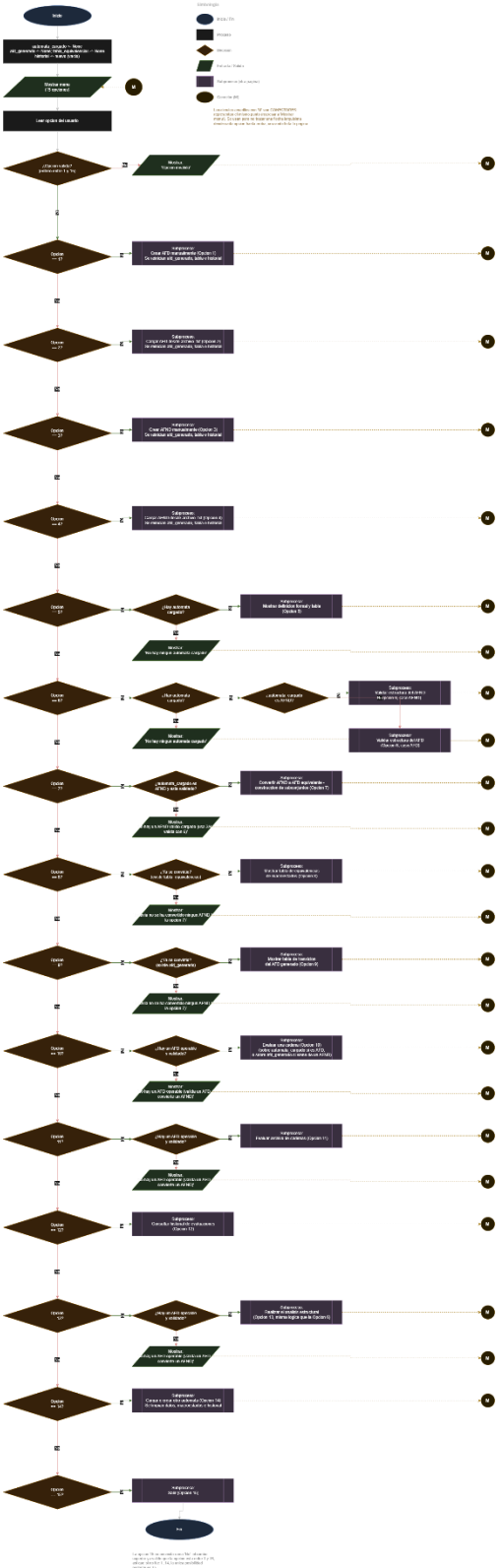


Diagrama Crear un AFD manualmente

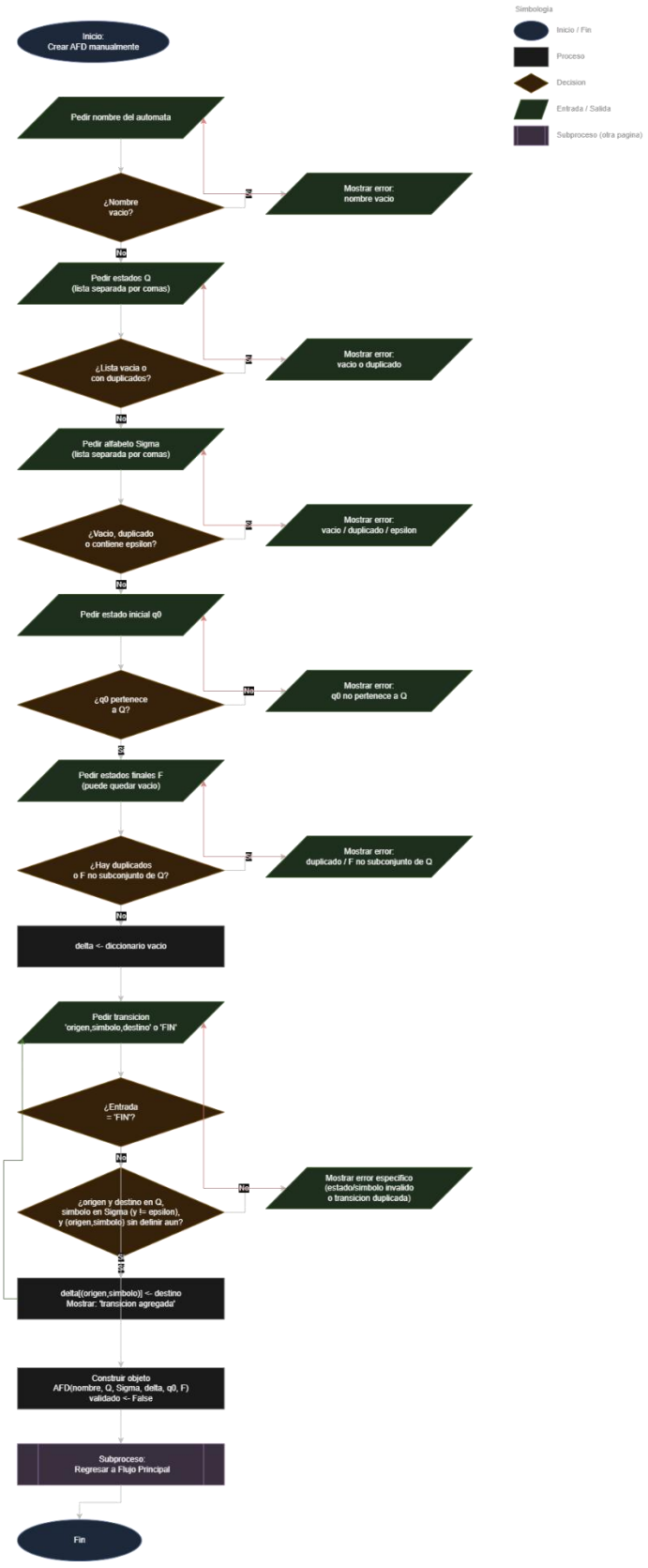


Diagrama Cargar un AFD desde archivo .txt

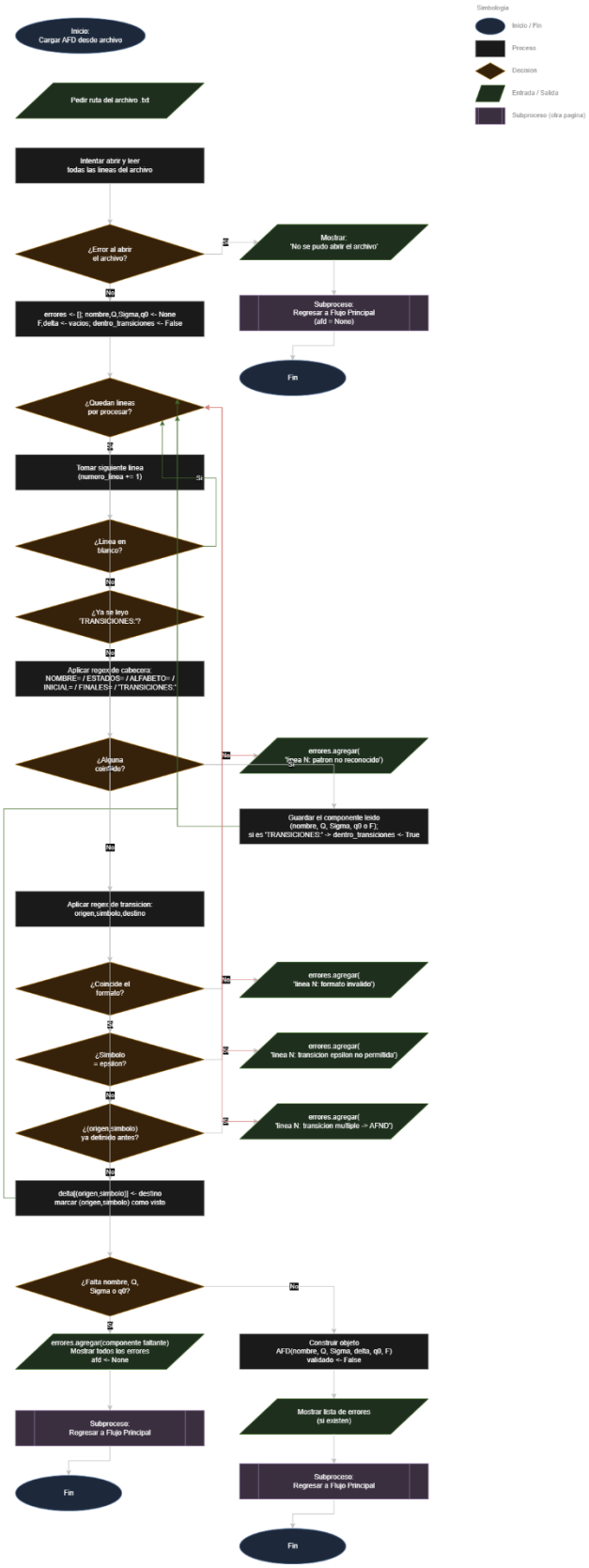


Diagrama Crear un AFND manualmente

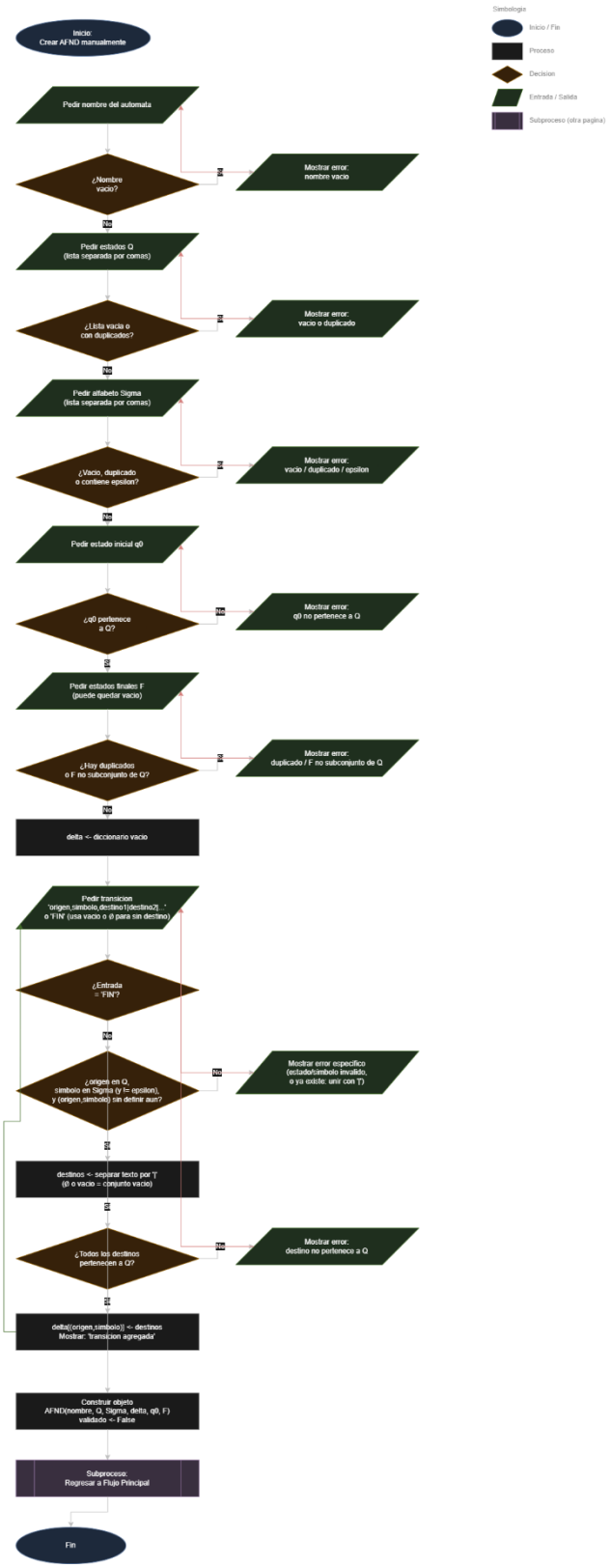


Diagrama Cargar un AFND desde archivo .txt

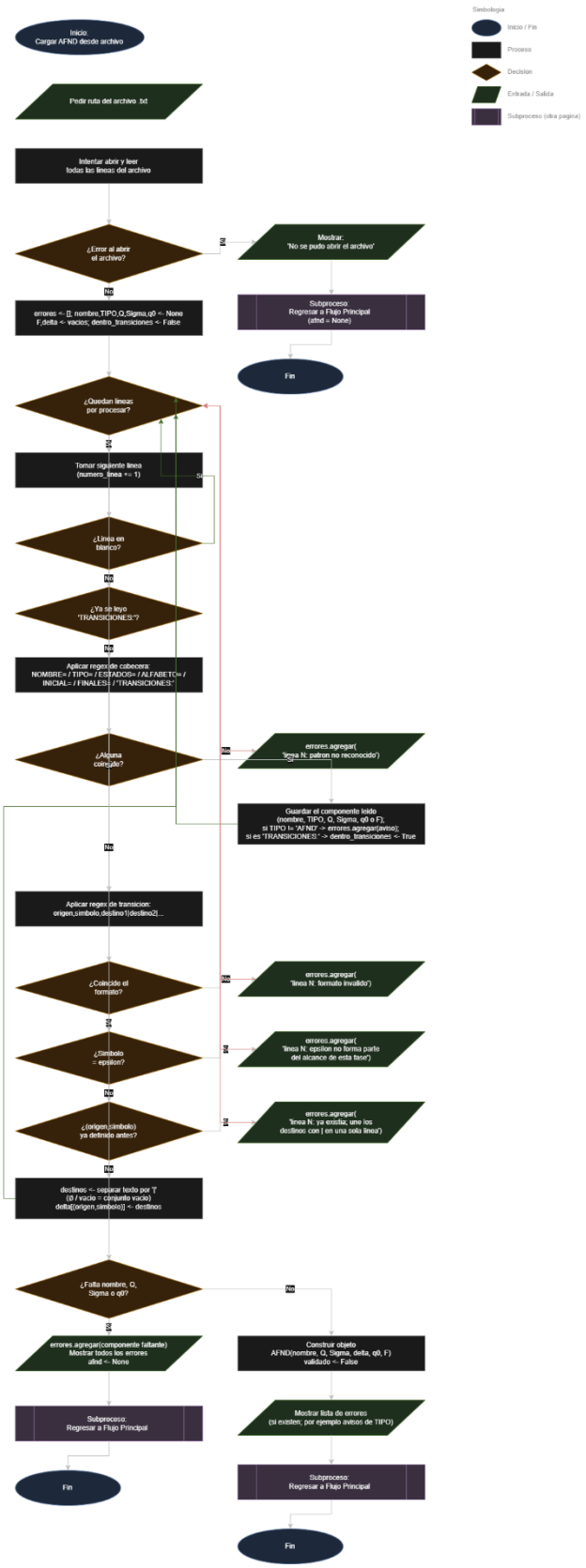


Diagrama Mostrar la definición formal y la tabla del autómata cargado

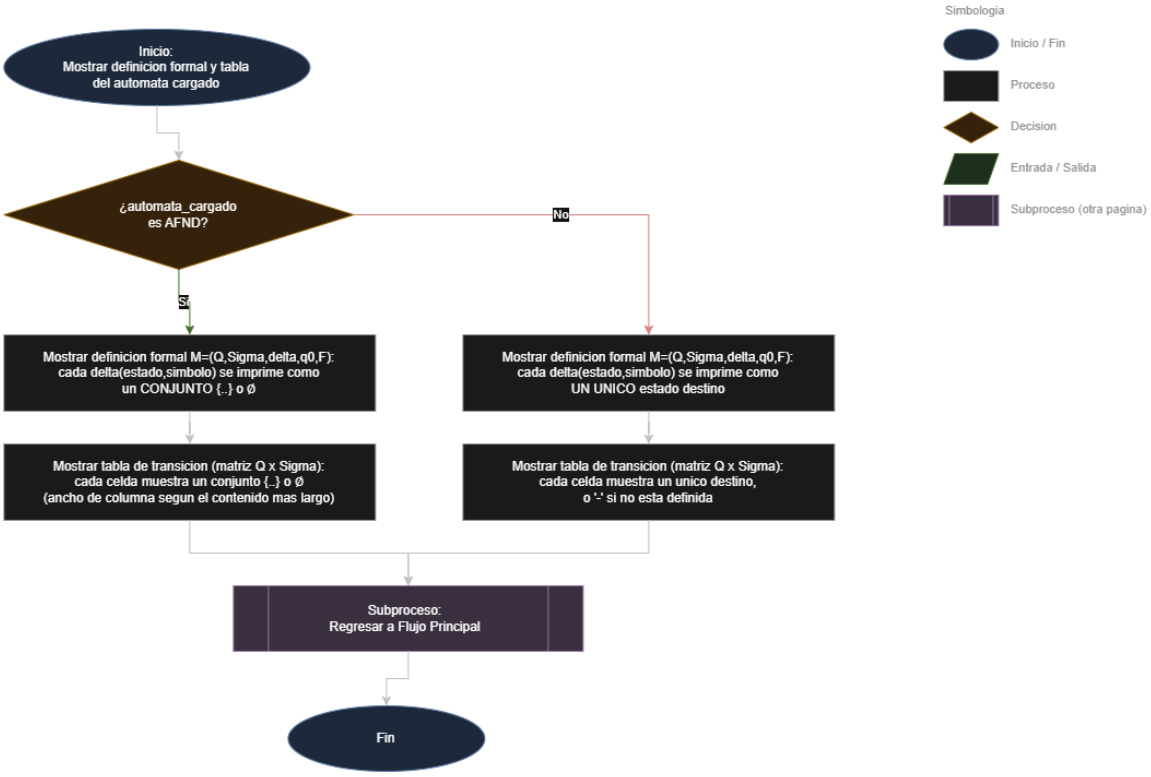
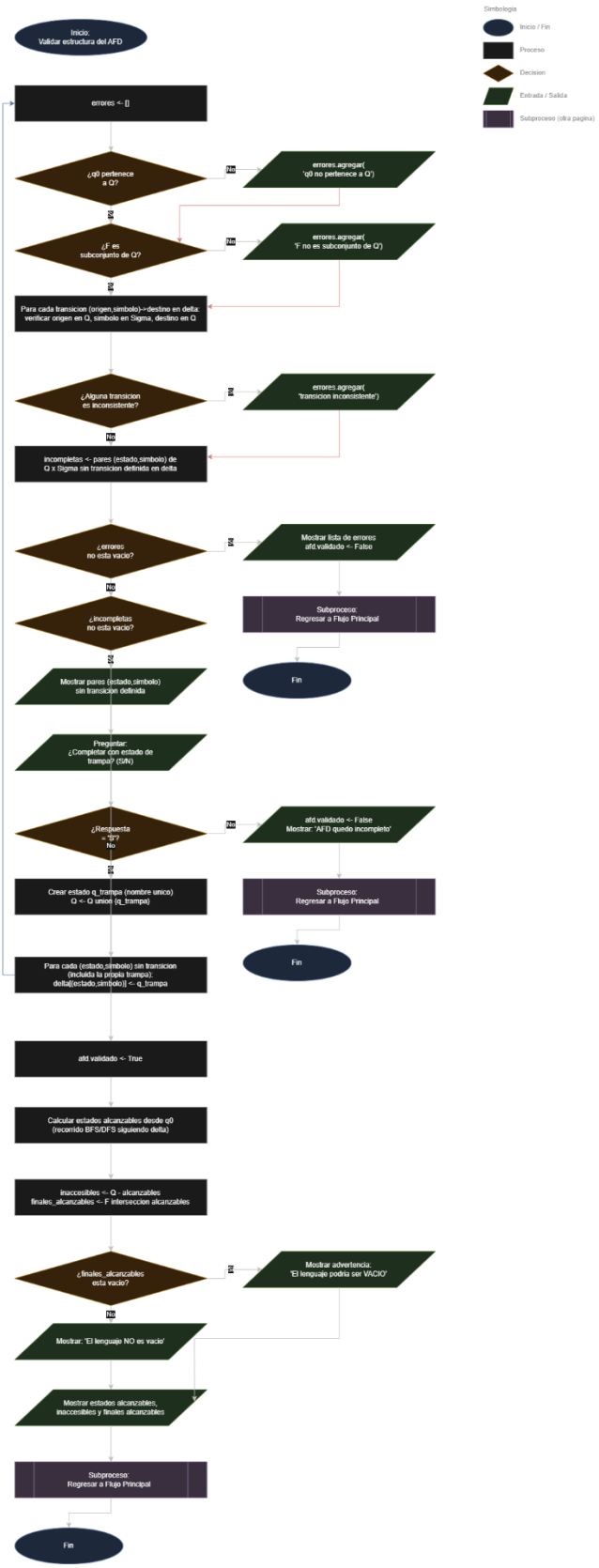


Diagrama Validar la estructura del autómata



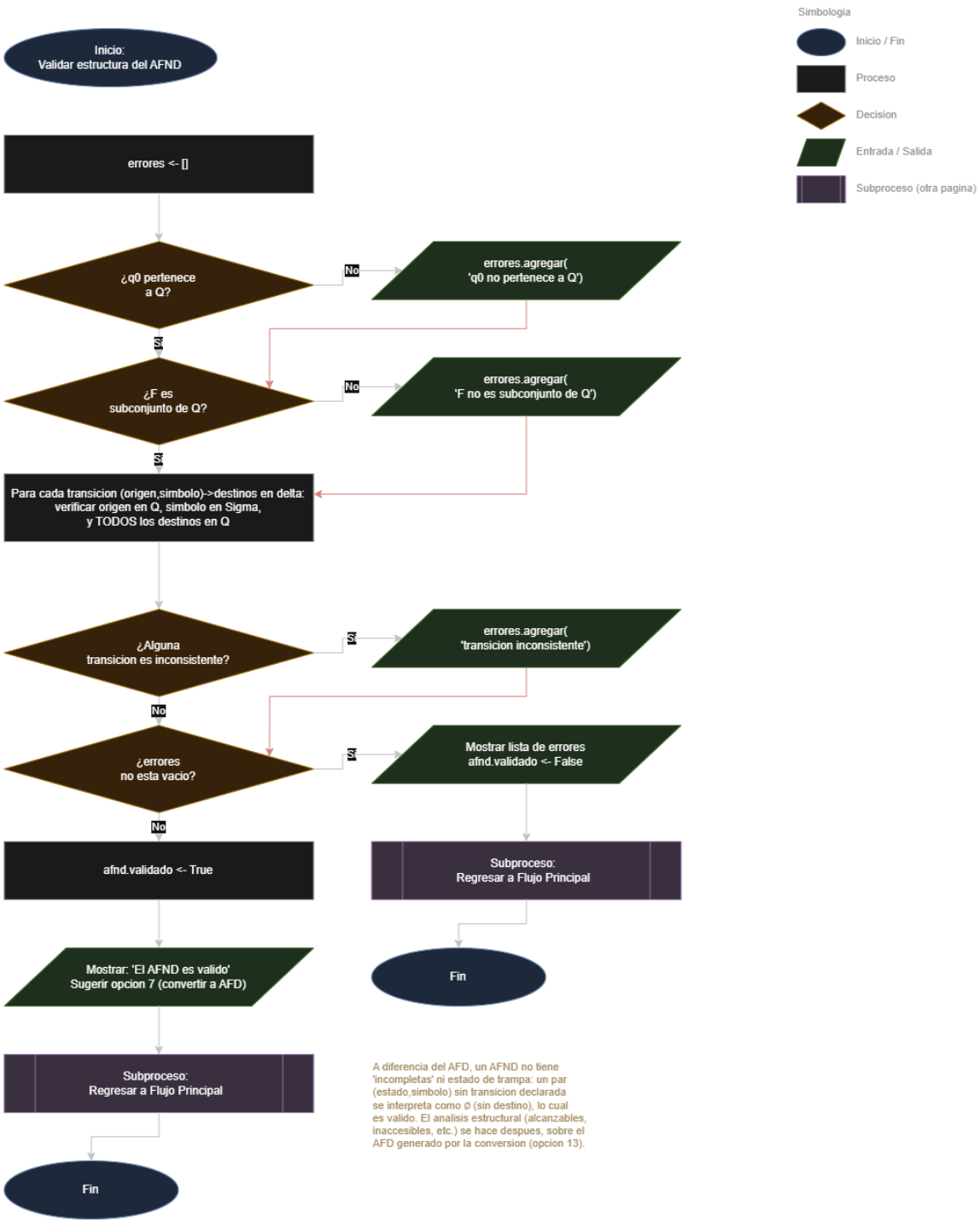

Inicio:
Validar estructura del AFND
errores <- []
¿q0 pertenece
a Q?
No
errores.agregar(
'q0 no pertenece a Q')
Si
¿F es
subconjunto de Q?
No
errores.agregar(
'F no es subconjunto de Q')
Si
Para cada transicion (origen,simbolo)->destinos en delta:
verificar origen en Q, simbolo en Sigma,
y TODOS los destinos en Q
¿Alguna
transicion es inconsistente?
Si
errores.agregar(
'transicion inconsistente')
No
¿errores
no esta vacio?
Si
Mostrar lista de errores
afnd.validado <- False
No
afnd.validado <- True
Subproceso:
Regresar a Flujo Principal
Mostrar: 'El AFND es valido'
Sugerir opcion 7 (convertir a AFD)
Fin
Subproceso:
Regresar a Flujo Principal
Fin
Simbologia
Inicio / Fin
Proceso
Decision
Entrada / Salida
Subproceso (otra pagina)
A diferencia del AFD, un AFND no tiene
'incompletas' ni estado de trampa: un par
(estado,simbolo) sin transicion declarada
se interpreta como Ø (sin destino), lo cual
es valido. El analisis estructural (alcanzables,
inaccesibles, etc.) se hace despues, sobre el
AFD generado por la conversion (opcion 13).

Diagrama Convertir el AFND cargado en un AFD equivalente

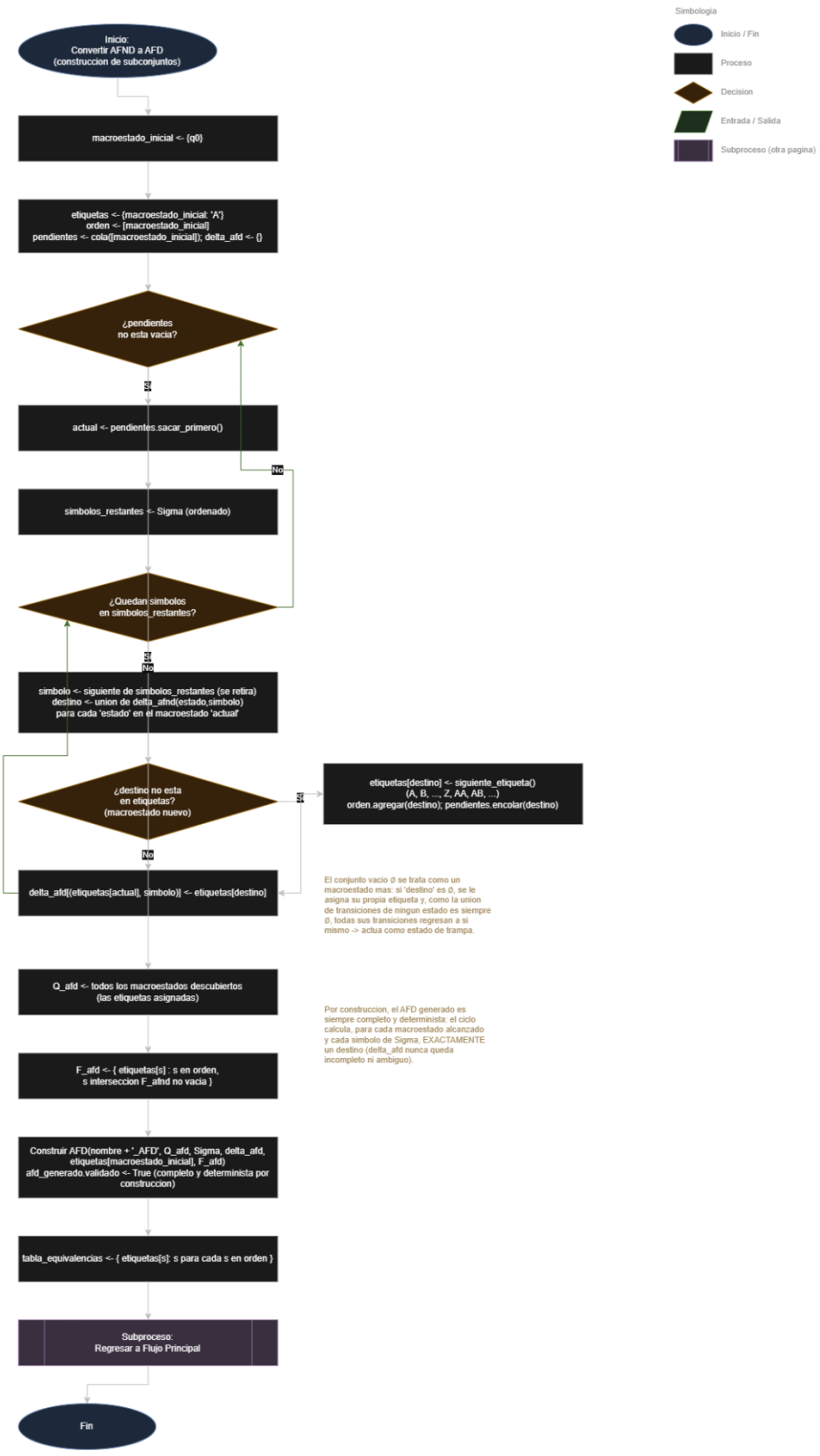
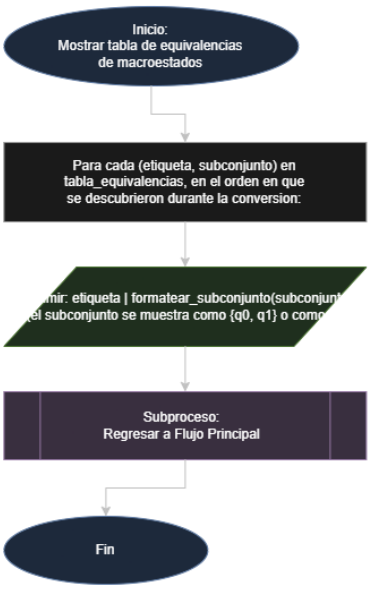


Diagrama Mostrar la tabla de equivalencias de macroestados



Esta tabla solo existe despues de convertir un AFND (opcion 7): relaciona cada macroestado del AFD generado (A, B, C, ...) con el subconjunto de estados del AFND que representa.

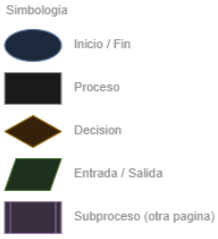


Diagrama Mostrar la tabla de transición del AFD generado



Diagrama Evaluar una cadena

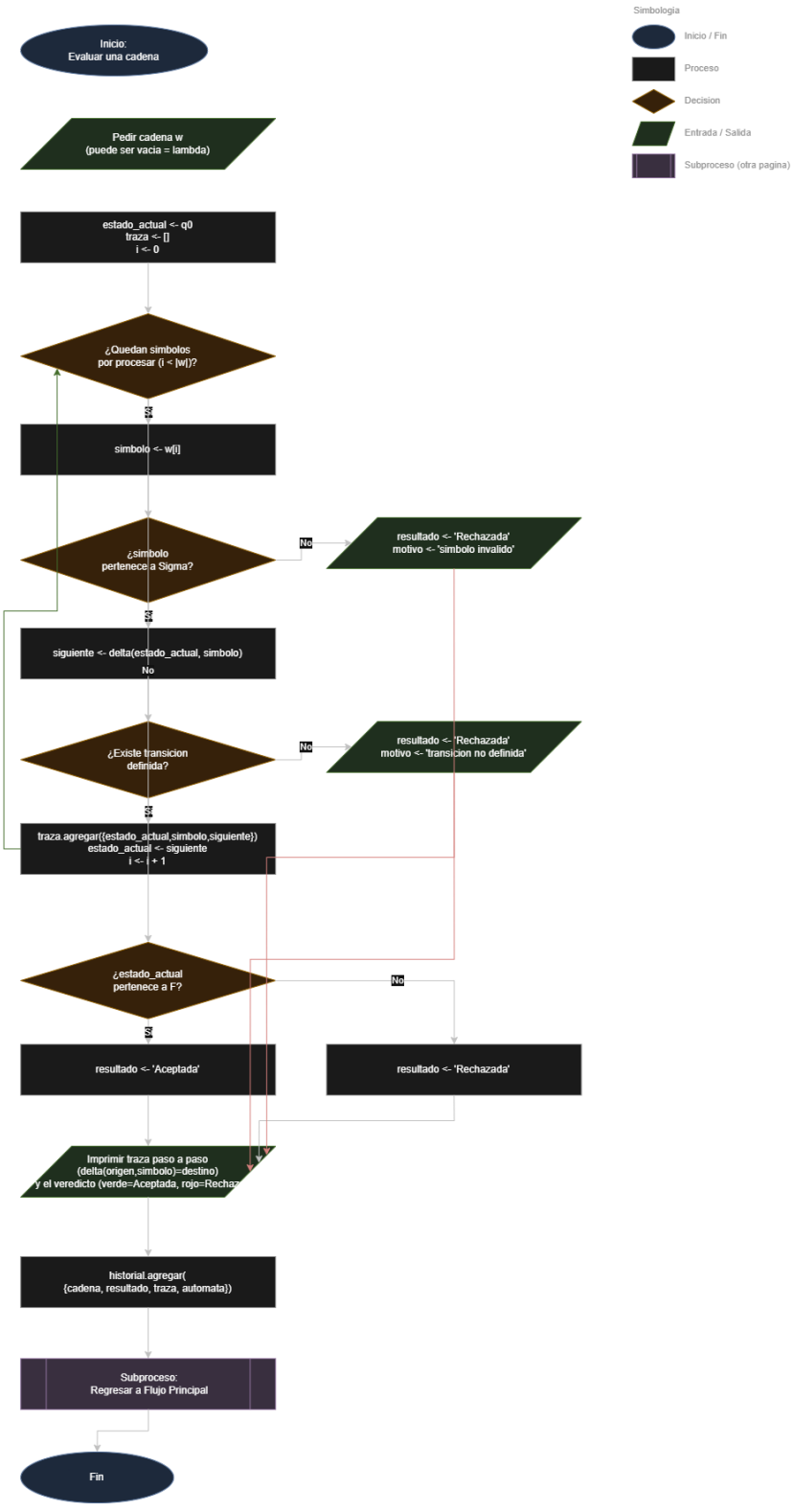


Diagrama Evaluar un archivo de cadenas

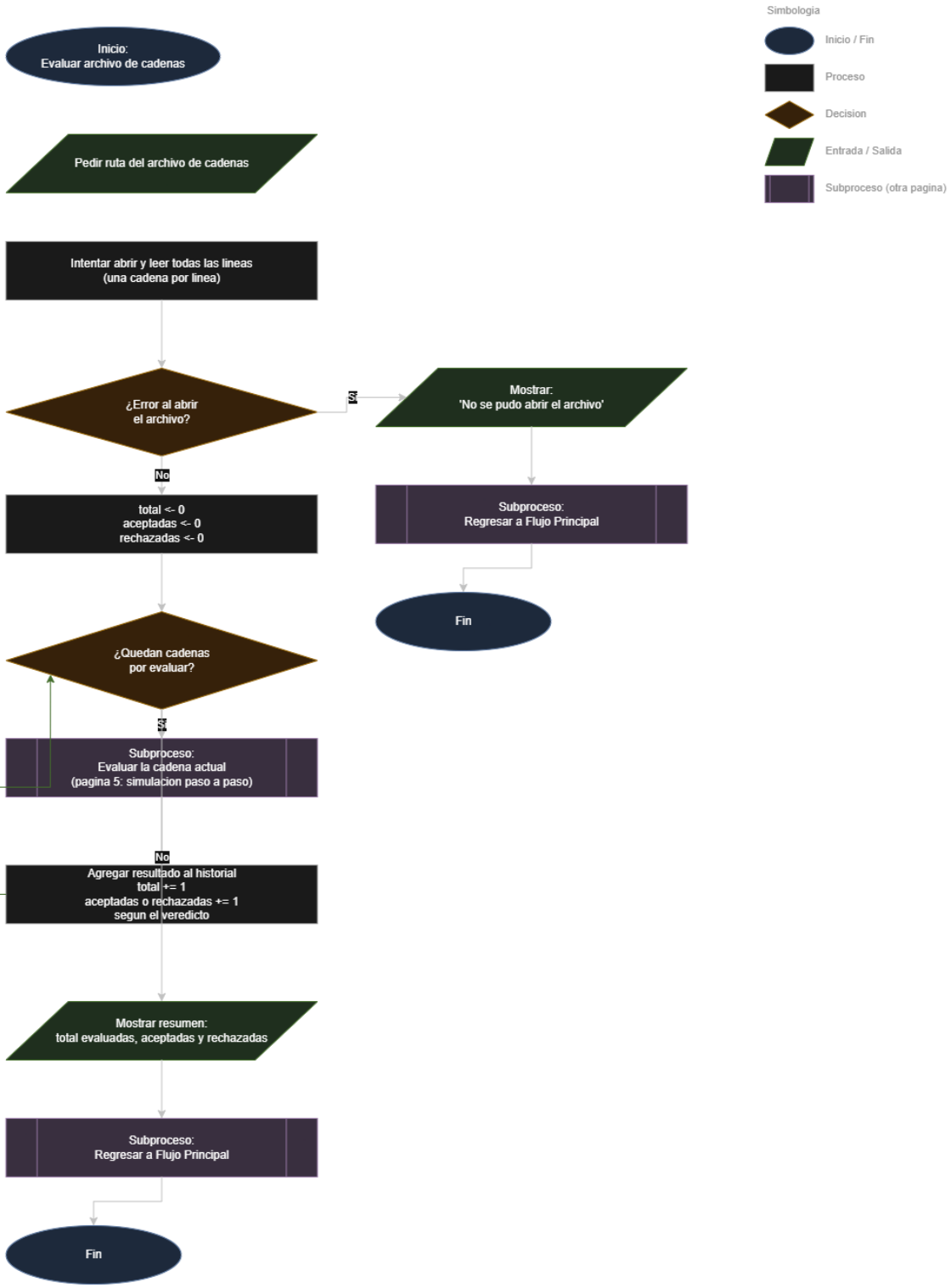


Diagrama Consultar el historial de evaluaciones

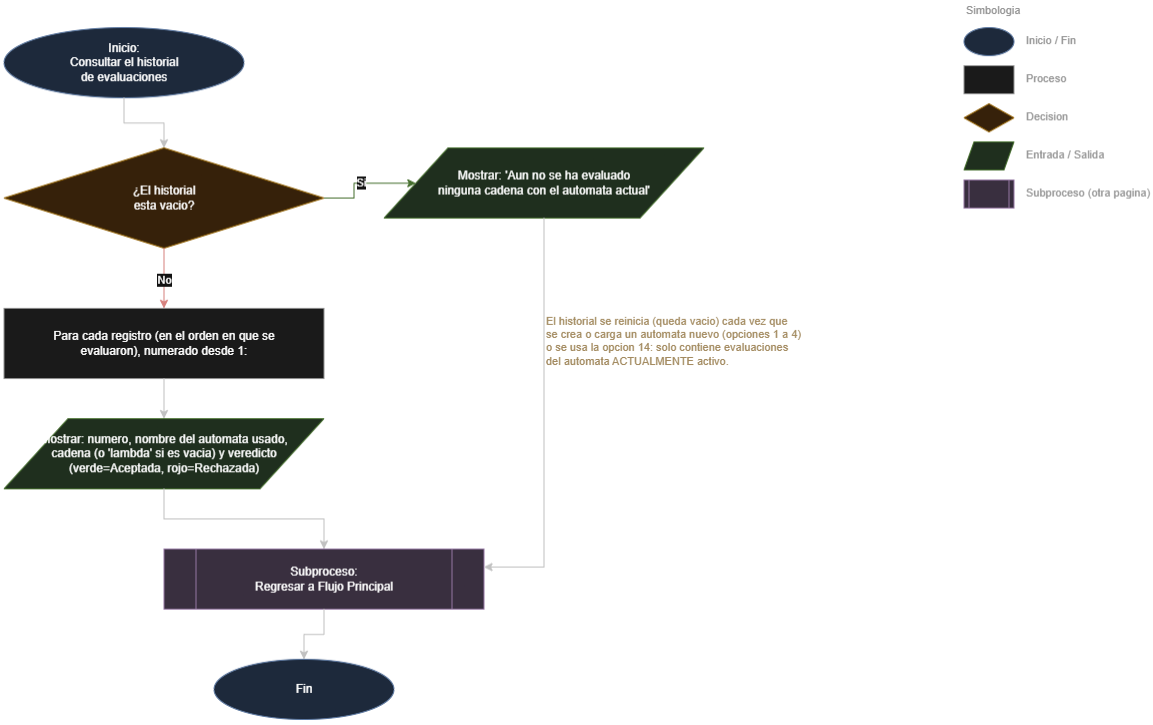


Diagrama Realizar el análisis estructural

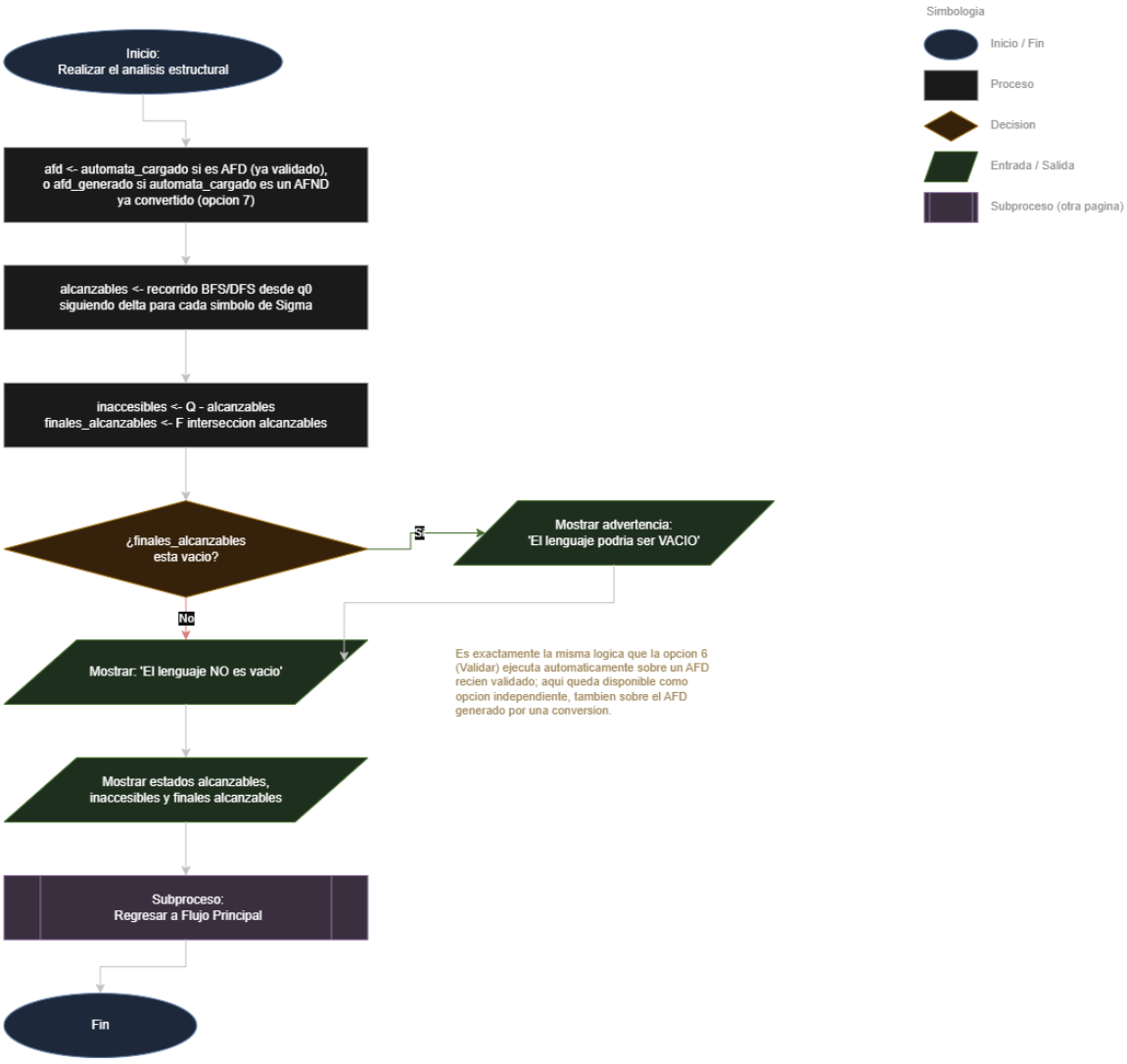


Diagrama Cargar o crear otro autómata

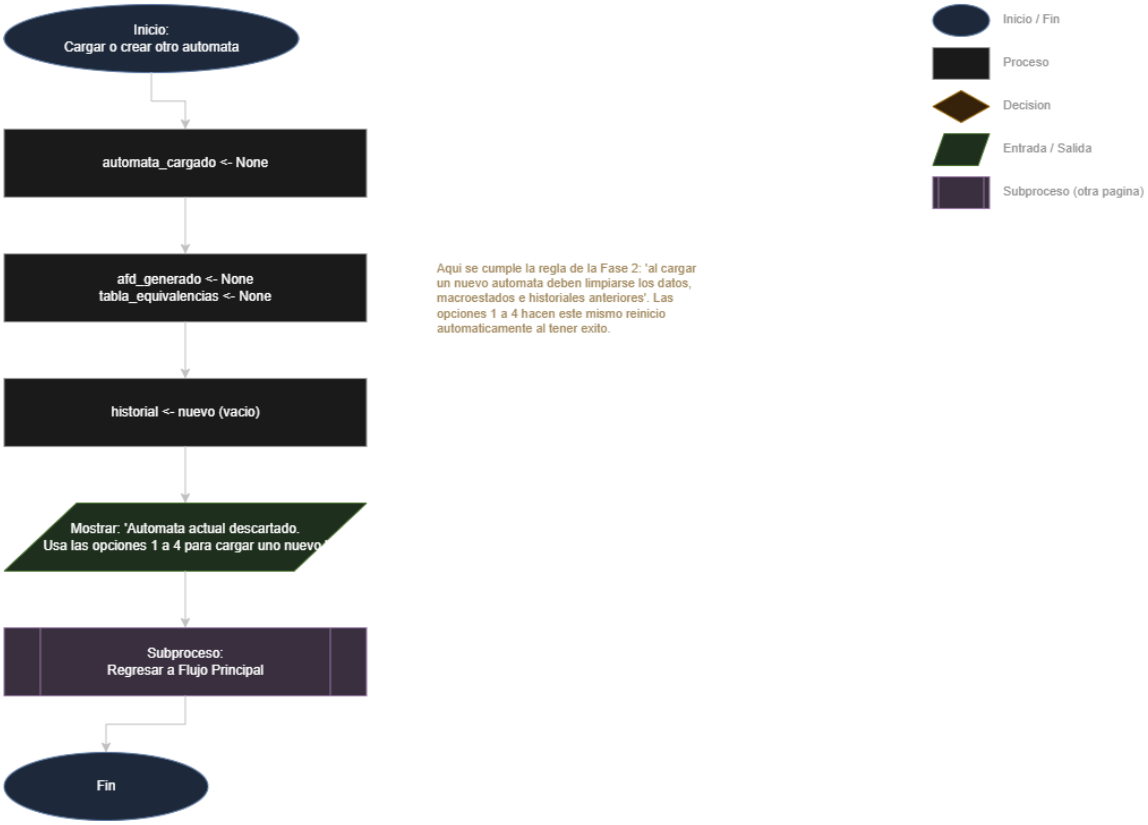
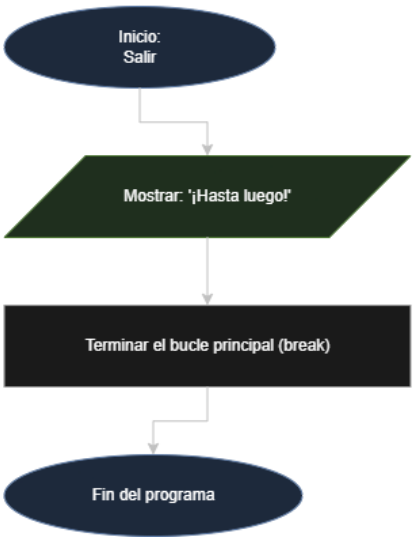
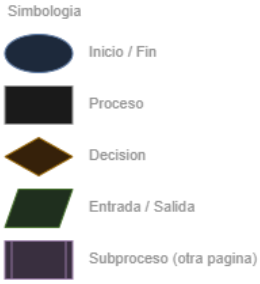


Diagrama Salir





Casos de prueba

Los archivos de prueba se encuentran en la carpeta ejemplos. La siguiente tabla resume cada caso obligatorio del enunciado, los pasos para reproducirlo, el resultado esperado y el resultado obtenido al ejecutar el programa.

#	Caso	Archivo y pasos	Resultado esperado	Resultado obtenido
1	AFD válido y completo (compatibilidad con la Fase 1)	afd_ab.txt: opciones 2, 6 y 11 con cadenas.txt	Válido, determinista y completo, sin estados inaccesibles. aab y ab aceptadas; λ, b y abz rechazadas	Coincide: 2 aceptadas y 3 rechazadas (abz por símbolo fuera de Σ)

#	Caso	Archivo y pasos	Resultado esperado	Resultado obtenido
2	AFD incompleto y estado de trampa	afd_incompleto.txt: opciones 2 y 6, responder S	Reporta 4 transiciones faltantes ((q2,a), (q2,b), (q3,a), (q3,b)); al aceptar agrega q_trampa y queda completo; q3 es inaccesible	Coincide
3	Definición inválida de AFD	afd_malformado.txt: opciones 2 y 6	5 errores de lectura con número de línea (estados duplicados, ε en Σ, línea no reconocida, transición múltiple, transición incompleta) y F ⊄ Q por q9; la opción 6 indica que NO es válido	Coincide
4	AFND con dos destinos para un mismo par	afnd_dos_destinos.txt: opciones 4, 5, 6, 7 y 8	La tabla del AFND muestra δ(q0,a) = {q0, q1}; la conversión genera A={q0}, B={q0,q1}, C={q0,q2}, D={q0,q1,q2}; finales C y D	Coincide
5	AFND con al menos tres macroestados	afnd_tres_macroestados.txt: opciones 4, 6, 7 y 8	4 macroestados: A={p0}, B={p0,p1}, C={p0,p1,p2}, D={p0,p2}; finales C y D	Coincide
6	Conversión que alcanza ∅	afnd_vacio.txt: opciones 4, 6, 7, 8 y 9	A={r0} (inicial y final) y B=∅ marcado como estado de trampa; δ(A,b)=B, δ(B,a)=δ(B,b)=B	Coincide
7	Macroestado final con estados finales y no finales	afnd_macroestado_mixto.txt: opciones 4, 6, 7 y 8	B={m0, m1} es final porque contiene m1, aunque m0 no es final	Coincide
8	Cadena aceptada	afnd_dos_destinos.txt convertido: opción 10 con aab	AFD: A→B→B→C, Aceptada. AFND: {q0}→{q0,q1}→{q0,q1}→{q0,q2}, Aceptada	Coincide (COINCIDEN)
9	Cadena rechazada	afnd_dos_destinos.txt convertido: opción 10 con ba	AFD: A→A→B, Rechazada. AFND: {q0}→{q0}→{q0,q1}, Rechazada	Coincide (COINCIDEN)
10	Cadena vacía (λ)	afnd_dos_destinos.txt convertido: opción 10 sin escribir nada	No se procesa ningún símbolo; Rechazada porque A={q0} no es final	Coincide (COINCIDEN)
11	Cadena con símbolos fuera del alfabeto	afnd_dos_destinos.txt convertido: opción 10 con abz	Rechazada con el motivo "El símbolo 'z' no pertenece al alfabeto" en el AFD y en el AFND	Coincide (COINCIDEN)
12	Archivo con errores sintácticos	afnd_errores_sintacticos.txt: opciones 4, 6 y 7	6 errores con número de línea (4 transiciones con formato inválido, destinos q0||q1 y transición ε); la opción 6 indica que NO es válido y la 7 no permite convertir	Coincide
13	Archivo con estados o símbolos inexistentes	afnd_estados_inexistentes.txt: opciones 4, 6 y 7	La opción 6 reporta F ⊄ Q (q2), destino q3 ∉ Q y símbolo c ∉ Σ; la opción 7 no permite convertir	Coincide
14	Comparación entre el AFND y el AFD generado	afnd_tres_macroestados.txt convertido: opción 11 con cadenas_afnd.txt	Cada cadena muestra el veredicto del AFD y del AFND; el resumen indica 7/7 cadenas	Coincide

#	Caso	Archivo y pasos	Resultado esperado	Resultado obtenido
			coinciden (aab y aaab aceptadas)	
15	Archivo de AFND cargado como AFD	afnd_dos_destinos.txt con la opción 2	Errores: TIPO=AFND, transición con varios destinos (|) y ∅ no válido en un AFD	Coincide
16	Reglas de operación y entradas inválidas	Opciones 7 o 10 sin autómata, 10 sobre un AFND sin convertir, "abc" o 99 en el menú, ruta inexistente	Mensaje claro en cada caso, sin cerrar el programa	Coincide